

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA

Curso: Cálculo II

Docente: Ing. José Alfredo González

PROYECTO

Emulador de Volúmenes de Sólidos de Revolución

Métodos del Disco, Arandelas y Capas Cilíndricas

Informe escrito

Integrantes:

[Nombre completo del integrante 1] — Carné [0000-00-00000]

[Nombre completo del integrante 2] — Carné [0000-00-00000]

[Nombre completo del integrante 3] — Carné [0000-00-00000]

Sección: [ ]

Guatemala, 25 de octubre de 2026

Índice

1. Introducción 4
2. Objetivos 5
2.1 Objetivo general 5
2.2 Objetivos específicos 5
3. Marco teórico 6
3.1 La integral definida 6
3.2 Sólidos de revolución 6
3.3 Método del disco 6
3.4 Método de arandelas 6
3.5 Método de capas cilíndricas 7
3.6 Compatibilidad entre método y eje 7
3.7 Integración numérica: regla de Simpson 1/3 compuesta 7
3.8 Volumen acumulado 8
3.9 Intersección de curvas: método de bisección 8
4. Justificación 9
5. Resolución manual del problema 10
a) Puntos de intersección 10
b) Función superior y función inferior 10
c) Deducción de la ecuación del volumen 10
d) Cálculo del volumen 11
e) Esquema de la región, curvas, límites y eje de rotación 12
5.1 Resolución manual de las configuraciones de validación 13
6. Desarrollo del simulador 14
6.1 Tecnologías utilizadas 14
6.2 Arquitectura del programa 14
6.3 Entrada de datos 14
6.4 Validación de datos 15
6.5 Cálculo del volumen 15
6.6 Comparación con el procedimiento manual 15
6.7 Gráficas 16
6.8 Exportación de datos 16
6.9 Ejecución del programa 16
6.10 Pruebas unitarias 16
7. Validación y análisis de resultados 17
7.1 Tabla de validación (sección 13) 17

7.2 Análisis del error numérico 17
7.3 Análisis del volumen acumulado....18
7.4 Correspondencia con la maqueta 19
8. Conclusiones 20
9. Recomendaciones 21
10. Bibliografía....22
11. Anexos....23
Anexo A. Código fuente....23
Anexo B. Capturas de pantalla del simulador 31
Anexo C. Archivo de datos exportado 33
Anexo D. Gráficas 34
Anexo E. Maqueta física....34

(Si el índice aparece vacío: en Word, clic derecho sobre él → «Actualizar campos» → «Actualizar toda la tabla».)

1. Introducción

El cálculo integral permite medir magnitudes que no pueden obtenerse con fórmulas geométricas elementales. Una de sus aplicaciones más visuales es el cálculo del volumen de un **sólido de revolución**: el cuerpo que se genera cuando una región plana gira 360° alrededor de una recta llamada eje de rotación. Piezas como ejes escalonados, toberas, recipientes, poleas o lentes se modelan exactamente de esta manera, por lo que el tema tiene una relación directa con el trabajo del ingeniero.

El presente informe documenta el desarrollo del proyecto «Emulador de Volúmenes de Sólidos de Revolución» del curso de Cálculo II. El problema central consiste en calcular el volumen del sólido generado al girar, alrededor del eje X, la región limitada por las curvas $y = x^2 + 1$ y $y = x + 3$, utilizando el **método de arandelas**. El resultado se obtiene primero mediante un procedimiento manual completo y luego se verifica con un simulador computacional desarrollado en **Java 21 con JavaFX**, que además implementa los métodos del disco y de capas cilíndricas.

El simulador valida los datos de entrada, construye la integral correspondiente al método elegido, la evalúa numéricamente con la regla de Simpson 1/3 compuesta, compara el resultado con el valor manual, genera la gráfica 2D de la región, la representación 3D del sólido y la curva de volumen acumulado V_acum(x), y exporta los datos a archivos CSV y TXT. Finalmente, se construyó una maqueta física con un motor de 5 V que, al hacer girar la región plana a alta velocidad, permite observar el sólido de revolución de forma tangible.

El documento se organiza así: objetivos, marco teórico, justificación, resolución manual del problema, desarrollo del simulador, validación y análisis de resultados, conclusiones, recomendaciones, bibliografía y anexos (código fuente, capturas de pantalla, datos exportados, gráficas y maqueta).

2. Objetivos

2.1 Objetivo general

Desarrollar un simulador computacional que calcule y represente gráficamente el volumen de sólidos de revolución mediante los métodos del disco, de arandelas y de capas cilíndricas, y validar sus resultados con la resolución manual del problema y con una maqueta física.

2.2 Objetivos específicos

1. Determinar analíticamente los puntos de intersección de $y = x^2 + 1$ y $y = x + 3$, identificar la función superior y la inferior y deducir la integral del volumen por el método de arandelas.
2. Calcular de forma exacta el volumen del sólido generado al girar la región alrededor del eje X.
3. Implementar en Java las fórmulas de los tres métodos, la validación de datos y la integración numérica por la regla de Simpson 1/3 compuesta.
4. Generar la gráfica 2D de la región, la gráfica 3D del sólido y la gráfica del volumen acumulado V_acum(x), y exportar los datos a CSV/TXT.
5. Comparar los resultados manuales con los computacionales en las cuatro configuraciones de prueba y cuantificar el error porcentual.
6. Construir una maqueta física, accionada por un motor de 5 V, que represente el sólido, su eje de rotación, la región generadora y los límites de integración.

3. Marco teórico

3.1 La integral definida

Si una función h es continua en [a, b], se divide el intervalo en n subintervalos de ancho Δx = (b − a)/n y se elige un punto x_i^* en cada uno. La integral definida es el límite de las sumas de Riemann:

$$\int_a^b h(x)\,dx = \lim_{n\to\infty} \sum_{i=1}^{n} h(x_i^*)\,\Delta x$$

El **Teorema Fundamental del Cálculo** permite evaluarla con una antiderivada H (H′ = h): $\int_a^b h(x)\,dx = H(b) - H(a)$. La misma idea —dividir, aproximar cada pedazo y sumar— es la que da origen a las fórmulas de volumen: si se conoce el área A(x) de cada sección transversal perpendicular al eje, el volumen es $V = \int_a^b A(x)\,dx$ (Stewart et al., 2021).

3.2 Sólidos de revolución

Un sólido de revolución se obtiene al girar una región plana alrededor de una recta del mismo plano. Cada punto de la región a una distancia ρ del eje describe una circunferencia de radio ρ. Por eso las secciones perpendiculares al eje son círculos (discos) o coronas circulares (arandelas), y las capas paralelas al eje son cilindros.

3.3 Método del disco

Si la región está entre la curva y = f(x) ≥ 0 y el eje X, sobre [a, b], cada rebanada perpendicular al eje X es un disco de radio f(x) y grosor dx. Su volumen es π[f(x)]² dx, y al sumar:

$$V = \pi \int_a^b [f(x)]^2\,dx$$

3.4 Método de arandelas

Si la región está entre dos curvas f(x) ≥ g(x) ≥ 0 y no toca el eje de rotación, cada rebanada es una arandela (disco con un agujero). El radio exterior es R(x) = f(x) y el radio interior es r(x) = g(x). El área de la arandela es π(R² − r²), por lo que:

$$V = \pi \int_a^b \left([f(x)]^2 - [g(x)]^2 \right) dx$$

Es importante notar que se restan los **cuadrados** de los radios, no los radios: π(R² − r²) ≠ π(R − r)². Este es el error más común en el método.

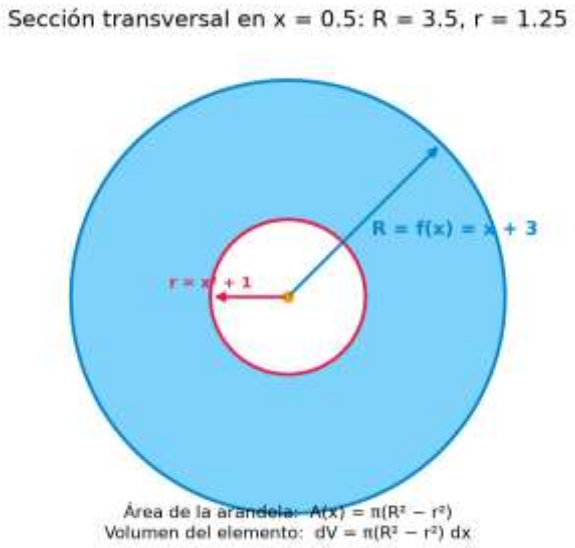


Figura 1. Sección transversal típica (arandela) del sólido del ejercicio principal, en x = 0.5.

3.5 Método de capas cilíndricas

Cuando la región gira alrededor del eje Y y está descrita en términos de x, conviene descomponerla en franjas verticales de ancho dx. Al girar, cada franja produce una capa cilíndrica de radio x, altura f(x) − g(x) y espesor dx, cuyo volumen es 2πx[f(x) − g(x)] dx. Para 0 ≤ a < b:

$$V = 2\pi \int_a^b x\,(\,f(x) - g(x)\,)\,dx$$

3.6 Compatibilidad entre método y eje

En el alcance de este proyecto, cada método se usa con un eje determinado. Los discos y arandelas integran secciones perpendiculares al eje de integración (x), por lo que el eje debe ser el eje X; las capas integran cilindros concéntricos al eje Y con radio x, por lo que exigen a ≥ 0 para que el radio no sea negativo.

Método	Eje permitido	Condición sobre la región	Fórmula
Disco	Eje X	f(x) ≥ g(x) = 0	$V = \pi\int_a^b f^2\,dx$
Arandelas	Eje X	f(x) ≥ g(x) ≥ 0	$V = \pi\int_a^b (f^2 - g^2)\,dx$
Capas cilíndricas	Eje Y	0 ≤ a < b, f(x) ≥ g(x)	$V = 2\pi\int_a^b x(f - g)\,dx$

Tabla 1. Combinaciones método–eje admitidas por el simulador.

3.7 Integración numérica: regla de Simpson 1/3 compuesta

Para que el simulador funcione con cualquier f(x) y g(x), la integral se evalúa numéricamente. La regla de Simpson 1/3 aproxima la función en cada par de subintervalos por una parábola. Con n subintervalos (n par) y h = (b − a)/n:

$$\int_a^b h(x)\,dx \approx (h/3)\,[\,h(x_0) + 4h(x_1) + 2h(x_2) + 4h(x_3) + \ldots + 4h(x_{n-1}) + h(x_n)\,]$$

Su error de truncamiento es $E = -(b - a)\, h^4\, h^{(4)}(\xi) / 180$ para algún ξ en (a, b) (Burden et al., 2017). En consecuencia, la regla es **exacta para polinomios de grado ≤ 3** y su error disminuye con la cuarta potencia de h: al duplicar n el error se reduce aproximadamente 16 veces. El simulador usa n = 800 para el volumen total.

3.8 Volumen acumulado

Se define el volumen acumulado como el volumen generado desde el límite inferior a hasta una posición variable x:

$$\mathrm{V_acum}(x) = \int_a^x A(t)\, dt, \quad \text{con } \mathrm{V_acum}(a) = 0 \text{ y } \mathrm{V_acum}(b) = V$$

Por el Teorema Fundamental del Cálculo, $dV_acum/dx = A(x)$: la pendiente de la curva de volumen acumulado en cada punto es exactamente el área de la sección transversal en ese punto. Como $A(x) \geq 0$, la función es siempre creciente; crece más rápido donde las secciones son más grandes.

3.9 Intersección de curvas: método de bisección

Los límites de integración son, en general, las abscisas donde $f(x) = g(x)$. El simulador los obtiene buscando cambios de signo de $h(x) = f(x) - g(x)$ en una malla fina de $[-10, 10]$ y refinando cada uno con el método de bisección, que reduce a la mitad el intervalo que contiene la raíz en cada iteración (Chapra y Canale, 2015).

4. Justificación

El cálculo de volúmenes de revolución es un contenido fundamental de Cálculo II y una herramienta de uso cotidiano en ingeniería: permite estimar la capacidad de tanques y recipientes, la masa de piezas torneadas, la cantidad de material para fabricar un componente o el desplazamiento de un cuerpo sumergido. Sin embargo, los estudiantes suelen tener dificultad para visualizar cómo una región plana se transforma en un sólido y por qué la integral describe su volumen.

Este proyecto se justifica porque integra tres formas complementarias de abordar el problema:

- **Analítica:** la resolución manual asegura el dominio de los conceptos (intersecciones, radios, planteamiento y evaluación de la integral).
- **Computacional:** el simulador permite comprobar resultados, explorar otras funciones e intervalos en segundos y comprender la integración numérica, que es la que se usa en la práctica cuando no existe una antiderivada sencilla.
- **Física:** la maqueta convierte el concepto abstracto en un objeto observable; al girar la región con el motor se aprecia directamente el sólido y el hueco que justifica el uso de arandelas.

Además, desarrollar el simulador fortalece competencias de programación, validación de datos, visualización y documentación técnica, todas necesarias en el ejercicio profesional de la ingeniería.

5. Resolución manual del problema

Enunciado. Calcule el volumen del sólido generado al girar la región limitada por $y = x^2 + 1$ y $y = x + 3$ alrededor del eje X, utilizando el método de arandelas.
Datos: $y_1 = x^2 + 1$, $y_2 = x + 3$, eje de rotación: eje X ($y = 0$).

a) Puntos de intersección

Las curvas se cortan donde sus ordenadas son iguales:

$$x^2 + 1 = x + 3$$

$$x^2 - x - 2 = 0$$

$$(x - 2)(x + 1) = 0 \implies x = -1 \text{ o } x = 2$$

Sustituyendo en $y = x + 3$: para $x = -1$, $y = 2$; para $x = 2$, $y = 5$. Comprobación con la parábola: $(-1)^2 + 1 = 2$ ✓ y $2^2 + 1 = 5$ ✓.

Puntos de intersección: $P_1(-1, 2)$ y $P_2(2, 5)$. **Límites de integración:** $a = -1$, $b = 2$.

b) Función superior y función inferior

Se evalúa un punto de prueba dentro del intervalo, $x = 0$: $y_2(0) = 0 + 3 = 3$ y $y_1(0) = 0^2 + 1 = 1$. Como $3 > 1$, la recta está por encima. De forma general:

$$(x + 3) - (x^2 + 1) = -x^2 + x + 2 = -(x - 2)(x + 1) \geq 0 \quad \text{para } -1 \leq x \leq 2$$

Además, $x^2 + 1 \geq 1 > 0$, de modo que toda la región está por encima del eje X **sin tocarlo**. Por eso, al girar, queda un hueco en el centro del sólido y corresponde usar arandelas:

Elemento	Función	Papel en el método
Función superior f(x)	x + 3	Radio exterior R(x) = x + 3
Función inferior g(x)	$x^2 + 1$	Radio interior r(x) = $x^2 + 1$
Condición del proyecto	f(x) ≥ g(x) ≥ 0 en [−1, 2]	Se cumple (g(x) ≥ 1)

Tabla 2. Identificación de radios.

c) Deducción de la ecuación del volumen

Se corta el sólido con un plano perpendicular al eje X en la posición x. La sección es una arandela de radio exterior R(x) = x + 3 y radio interior $r(x) = x^2 + 1$, con área:

$$A(x) = \pi\,[R(x)^2 - r(x)^2] = \pi\,[\,(x + 3)^2 - (x^2 + 1)^2\,]$$

Una rebanada de grosor dx tiene volumen $dV = A(x)\,dx$. Sumando todas las rebanadas desde $x = -1$ hasta $x = 2$:

$$\mathbf{V = \pi \int_{-1}^{2} [\,(x + 3)^2 - (x^2 + 1)^2\,]\,dx}$$

Desarrollando los cuadrados:

$$(x+3)^2 = x^2 + 6x + 9$$

$$(x^2+1)^2 = x^4 + 2x^2 + 1$$

$$(x+3)^2 - (x^2+1)^2 = -x^4 - x^2 + 6x + 8$$

$$\mathbf{V = \pi \int_{-1}^{2} \left(-x^4 - x^2 + 6x + 8 \right) dx}$$

d) Cálculo del volumen

Una antiderivada del integrando es:

$$F(x) = -x^5/5 - x^3/3 + 3x^2 + 8x$$

Evaluando en los límites:

$$F(2) = -32/5 - 8/3 + 12 + 16 = (-96 - 40 + 420)/15 = 284/15$$

$$F(-1) = 1/5 + 1/3 + 3 - 8 = (3 + 5 + 45 - 120)/15 = -67/15$$

$$F(2) - F(-1) = 284/15 + 67/15 = 351/15 = 117/5 = 23.4$$

Por lo tanto:

$$\mathbf{V = 117\pi / 5 \approx 73.5133\ u^3}$$

Verificación de razonabilidad. El área de la región es $\int_{-1}^{2} (-x^2 + x + 2)\, dx = 4.5$ u². Por el teorema de Pappus, $V = 2\pi \cdot \bar{y} \cdot \text{Área}$, donde $\bar{y}$ es la distancia del centroide al eje: $73.5133 = 2\pi \cdot \bar{y} \cdot 4.5 \Longrightarrow \bar{y} = 2.6$, valor coherente porque la región se encuentra entre y = 1 y y = 5.

Volumen acumulado (forma cerrada)

Con la misma antiderivada, el volumen generado desde a = −1 hasta una posición x es:

$$V_acum(x) = \pi\,[\,F(x) - F(-1)\,] = \pi\,(\,-x^5/5 - x^3/3 + 3x^2 + 8x + 67/15\,),\quad -1 \le x \le 2$$

Por ejemplo, V_acum(0.5) = π(9.16875) ≈ 28.8045 u³ y V_acum(2) = 117π/5. Su derivada A(x) = π(−x⁴ − x² + 6x + 8) tiene A′(x) = π(−4x³ − 2x + 6) = 0 en x = 1, por lo que la sección más grande es A(1) = 12π ≈ 37.70 u², y en x = 1 la curva V_acum(x) tiene su punto de inflexión (crece con máxima rapidez).

e) Esquema de la región, curvas, límites y eje de rotación

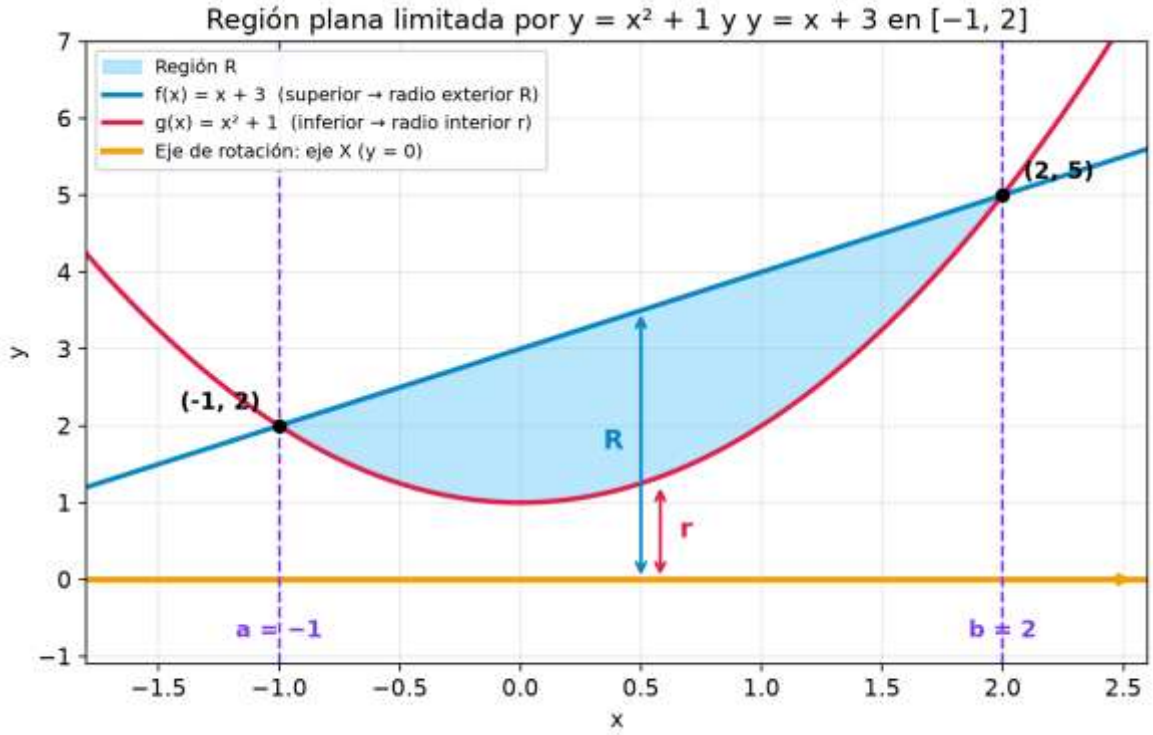


Figura 2. Región R limitada por y = x + 3 (radio exterior) y y = x² + 1 (radio interior) en [−1, 2]; eje de rotación y = 0.

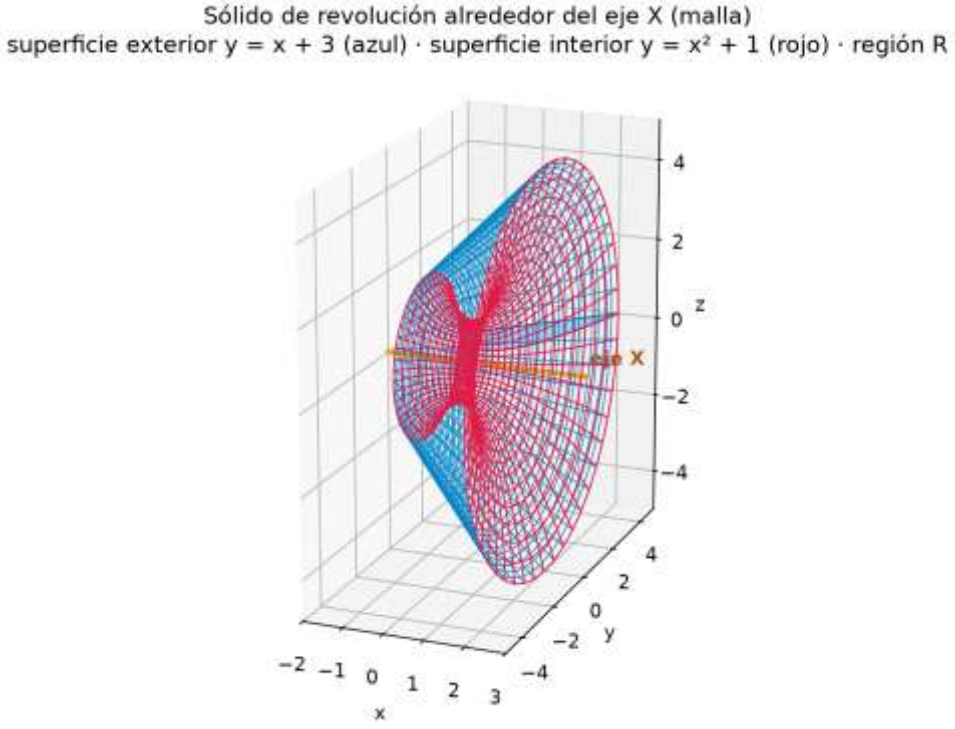


Figura 3. Sólido de revolución generado (malla: superficie exterior en azul, superficie interior —el hueco— en rojo; región R sombreada).

5.1 Resolución manual de las configuraciones de validación

La sección 13 del enunciado exige validar el simulador con tres configuraciones adicionales. Su resolución manual es la siguiente.

Configuración 1 — Discos, eje X: f(x) = √x, g(x) = 0, [0, 4]

$$V = \pi \int_0^4 (\sqrt{x})^2 \, dx = \pi \int_0^4 x \, dx = \pi \, [x^2/2]_0^4 = \pi(16/2) = 8\pi \approx 25.1327 \text{ u}^3$$

Configuración 2 — Arandelas, eje X: f(x) = √x, g(x) = x², [0, 1]

En [0, 1] se cumple √x ≥ x² ≥ 0 (se cortan en x = 0 y x = 1), por lo que R = √x y r = x²:

$$V = \pi \int_0^1 [\, (\sqrt{x})^2 - (x^2)^2 \,] \, dx = \pi \int_0^1 (x - x^4) \, dx = \pi \, [x^2/2 - x^5/5]_0^1 = \pi(1/2 - 1/5) = 3\pi/10 \approx 0.9425 \text{ u}^3$$

Configuración 3 — Capas cilíndricas, eje Y: f(x) = 2x, g(x) = x², [0, 2]

En [0, 2] se cumple 2x ≥ x² (se cortan en x = 0 y x = 2) y a = 0 ≥ 0. Radio de la capa: x; altura: 2x − x²:

$$V = 2\pi \int_0^2 x(2x - x^2) \, dx = 2\pi \int_0^2 (2x^2 - x^3) \, dx = 2\pi \, [2x^3/3 - x^4/4]_0^2 = 2\pi(16/3 - 4) = 8\pi/3 \approx 8.3776 \text{ u}^3$$

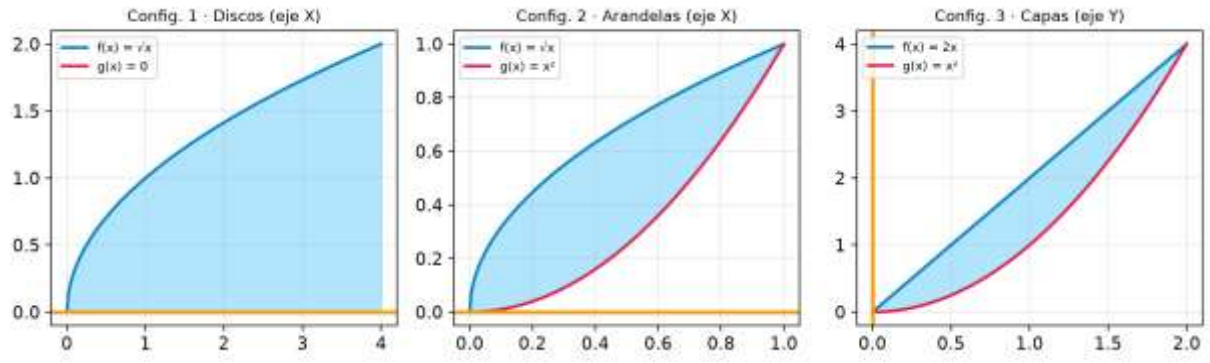


Figura 4. Regiones de las tres configuraciones de validación (eje de rotación en naranja).

6. Desarrollo del simulador

6.1 Tecnologías utilizadas

Herramienta	Versión	Uso en el proyecto
Java (JDK)	21 LTS	Lenguaje de programación del simulador
JavaFX	21.0.4	Interfaz gráfica, gráficas 2D (Canvas) y escena 3D (MeshView, PerspectiveCamera)
exp4j	0.4.8	Interpretación de las expresiones f(x) y g(x) escritas por el usuario
JUnit 5	5.10.2	Pruebas unitarias del motor de cálculo
Maven + jpackage	—	Compilación, empaquetado en JAR y generación del ejecutable .exe

Tabla 3. Tecnologías del simulador.

El enunciado ofrece una plantilla en R/RStudio, pero permite cualquier lenguaje. Se eligió Java porque produce un ejecutable de escritorio independiente (no requiere instalar Java ni R en la computadora del docente) y porque JavaFX incluye un motor 3D con el que se construyó la representación del sólido desde cero. Cada función de la plantilla tiene su equivalente en el código Java (Tabla 4).

6.2 Arquitectura del programa

Clase (paquete)	Responsabilidad	Equivale en la plantilla R a
FunctionEvaluator (engine)	Convierte el texto de f(x) y g(x) en funciones evaluables	f(), g()
Validator (engine)	Reglas de validación de la sección 6	validar_datos()
VolumeCalculator (engine)	Integrando de cada método, regla de Simpson y V_acum	calcular_volumen(), calcular_volumen_acumulado()
IntersectionFinder (engine)	Raíces de f(x) − g(x) por bisección	— (adicional)
RevolutionMeshBuilder (engine)	Malla triangular 3D del sólido	graficar_solido_3d()
DataExporter / ImageExporter (export)	Exportación CSV/TXT y PNG	exportar_datos(), ggsave()
MainView, Region2DCanvas, AccumCanvas, Solid3DView (ui)	Interfaz, gráfica 2D, gráfica V_acum y vista 3D	graficar_region(), graficar_volumen_acumulado(), programa principal
Presets (model)	Ejercicio principal y configuraciones 1–3 precargadas	—

Tabla 4. Clases principales y su correspondencia con la plantilla del enunciado.

6.3 Entrada de datos

Las funciones f(x) (superior) y g(x) (inferior) se definen en el código fuente como configuraciones predefinidas (clase Presets) y, además, pueden modificarse en la interfaz durante la defensa (se aceptan expresiones como x^2 + 1, sqrt(x), sin(x), exp(x), pi). El usuario ingresa los límites a y b —o los calcula automáticamente con el botón «Calcular intersecciones f(x) = g(x)»—, selecciona

el método de cálculo y el eje de rotación, e ingresa el volumen obtenido manualmente (acepta 73.5133, 73,5133 o la forma exacta 117*pi/5).

6.4 Validación de datos

Antes de calcular, la clase Validator revisa los datos y muestra mensajes claros. Si hay algún error, el volumen no se calcula.

Regla verificada	Mensaje mostrado
a y b son números	Los límites de integración a y b deben ser números válidos.
a < b	Debe cumplirse que a < b.
Disco/arandelas solo con eje X	Los métodos de discos y arandelas deben ser alrededor del eje X.
Capas solo con eje Y	El método de capas cilíndricas debe ser alrededor del eje Y.
Capas: 0 ≤ a	Para capas cilíndricas alrededor del eje Y debe cumplirse 0 <= a < b.
f y g definidas en [a, b] (201 puntos)	Las funciones no se pudieron evaluar en todo el intervalo [a, b].
f(x) ≥ g(x) en [a, b]	Debe cumplirse f(x) >= g(x) en todo el intervalo [a, b].
Disco/arandelas: g(x) ≥ 0	Para discos y arandelas, debe cumplirse f(x) >= g(x) >= 0 en [a, b].

Tabla 5. Validaciones implementadas (sección 6 del enunciado).

Además, en la interfaz el botón del eje incompatible se deshabilita al seleccionar el método, de modo que la combinación incorrecta no puede elegirse por accidente; si se fuerza, el validador la rechaza.

6.5 Cálculo del volumen

El método VolumeCalculator.buildIntegrand construye el integrando según el método elegido, con exactamente las mismas fórmulas del marco teórico y de la resolución manual:

Método	Integrando implementado (Java)
DISCOS	$\pi \cdot f(x)^2$
ARANDELAS	$\pi \cdot (f(x)^2 - g(x)^2)$
CAPAS	$2\pi \cdot x \cdot (f(x) - g(x))$

Tabla 6. Integrandos del código.

Ese integrando se evalúa con la regla de Simpson 1/3 compuesta (VolumeCalculator.simpsonIntegrate) con n = 800 subintervalos. El resultado se muestra con cuatro decimales y unidades cúbicas (u^3), junto con la integral planteada; por ejemplo: «73.5133 u^3 — $V = \pi \int ([f(x)]^2 - [g(x)]^2)\, dx$ en [−1.00, 2.00]».

6.6 Comparación con el procedimiento manual

El usuario escribe el volumen manual y el programa calcula la diferencia porcentual respecto del valor del simulador:

$$\text{Diferencia } \% = |\ \text{V_simulador} - \text{V_manual}\ |\ /\ |\ \text{V_simulador}\ | \times 100$$

Se informa la concordancia con los mismos criterios de la plantilla: < 0.5 % excelente concordancia; < 2 % diferencia pequeña; < 5 % diferencia moderada; ≥ 5 % diferencia significativa. También se contempla el caso en que ambos resultados son cero.

6.7 Gráficas

- **Gráfica 2D de la región:** curva superior f(x) en azul, curva inferior g(x) en rosa, región sombreada, líneas discontinuas en a y b, eje de rotación resaltado en naranja, ejes rotulados y escala automática.
- **Gráfica 3D del sólido:** la clase RevolutionMeshBuilder construye un perfil cerrado (curva exterior → tapa → curva interior → tapa) y lo barre alrededor del eje para formar una malla triangular. La vista permite rotar con el ratón, hacer zoom, mostrar un corte transversal en una posición x elegida (con el valor de A(x)), cortar el sólido por la mitad y animar la generación del sólido girando la región.
- **Volumen acumulado:** se grafica V_acum(x) desde a hasta b con 151 puntos, cada uno calculado con Simpson (n = 200) desde a hasta x; se marca el volumen total.

Todas las gráficas se pueden exportar a PNG con el botón «Exportar gráficas PNG».

6.8 Exportación de datos

Los botones «Exportar CSV» y «Exportar TXT» generan un archivo con las columnas x, f(x), g(x) y V_acum(x) (151 filas, 4 decimales). El CSV usa comas y el TXT tabulaciones. Un extracto se presenta en el Anexo C.

6.9 Ejecución del programa

- **Ejecutable de Windows:** abrir SimuladorSolidos\SimuladorSolidos.exe (no requiere Java instalado; copiar la carpeta completa).
- **JAR:** java -jar simulador-solidos-revolucion-1.0.0-shaded.jar (requiere Java 21).
- **Desde el código fuente:** mvn javafx:run (ejecutar), mvn test (pruebas), mvn package (compilar).

6.10 Pruebas unitarias

Se escribieron pruebas automáticas con JUnit 5 (VolumeCalculatorTest e InputAndIntersectionTest) que verifican los cuatro volúmenes de referencia, la búsqueda de intersecciones (por ejemplo, que para el ejercicio principal se obtengan x = −1 y x = 2) y que el validador rechace datos incorrectos.

7. Validación y análisis de resultados

7.1 Tabla de validación (sección 13)

Config.	Funciones	Intervalo	Método (eje)	Manual (u³)	Simulador (u³)	Error %
Principal	f = x + 3, g = x² + 1	[−1, 2]	Arandelas (X)	117π/5 = 73.5133	73.5133	< 0.0001 %
1	f = √x, g = 0	[0, 4]	Discos (X)	8π = 25.1327	25.1327	< 0.0001 %
2	f = √x, g = x²	[0, 1]	Arandelas (X)	3π/10 = 0.9425	0.9425	< 0.0001 %
3	f = 2x, g = x²	[0, 2]	Capas (Y)	8π/3 = 8.3776	8.3776	< 0.0001 %

Tabla 7. Comparación entre la resolución manual y el simulador.

En las cuatro configuraciones la diferencia entre el valor manual y el del simulador es menor a 0.0001 %, muy por debajo del umbral de 0.5 % que el programa considera «excelente concordancia». El simulador muestra la misma tabla en la pestaña «Tabla de Validación» (Anexo B, Figura B2) con el indicador «4/4 configuraciones validadas».

7.2 Análisis del error numérico

La pequeñísima diferencia del ejercicio principal no es un error de programación sino el error de truncamiento de la regla de Simpson. El integrando $-x^4 - x^2 + 6x + 8$ es de grado 4 y su cuarta derivada es constante (−24 por π), por lo que el error teórico puede calcularse exactamente:

$$|E| = (b - a)\, h^4 \cdot 24\pi \,/\, 180 = 3 \cdot (3/800)^4 \cdot 24\pi \,/\, 180 \approx 2.49 \times 10^{-10}\ u^3$$

El valor observado es |73.5132680937526 − 117π/5| ≈ 2.49e-10 u³, que coincide con la predicción teórica. En las configuraciones 1 y 3 los integrandos (πx y 2πx(2x − x²)) son polinomios de grado ≤ 3, para los cuales Simpson es exacto: la diferencia es solo de redondeo de la computadora (≈ 10^{-15}). En la configuración 2, π(x − x⁴) es de grado 4 y el error es del orden de 10^{-10}.

La Tabla 8 muestra la convergencia del método para el ejercicio principal al aumentar el número de subintervalos n:

n	h	V aproximado (u³)	Error absoluto	Error relativo
4	0.75000	73.11566027	3.98e-1	5.41e-1 %
10	0.30000	73.50308933	1.02e-2	1.38e-2 %
50	0.06000	73.51325181	1.63e-5	2.22e-5 %
200	0.01500	73.51326803	6.36e-8	8.65e-8 %
800	0.00375	73.51326809	2.49e-10	3.38e-10 %

Tabla 8. Convergencia de la regla de Simpson 1/3 compuesta (ejercicio principal).

Al multiplicar n por 4 (de 200 a 800) el error se divide entre $4^4 = 256$, exactamente lo que predice el término h^4. Con n = 800 el resultado ya es correcto en todas las cifras significativas que se reportan.

7.3 Análisis del volumen acumulado

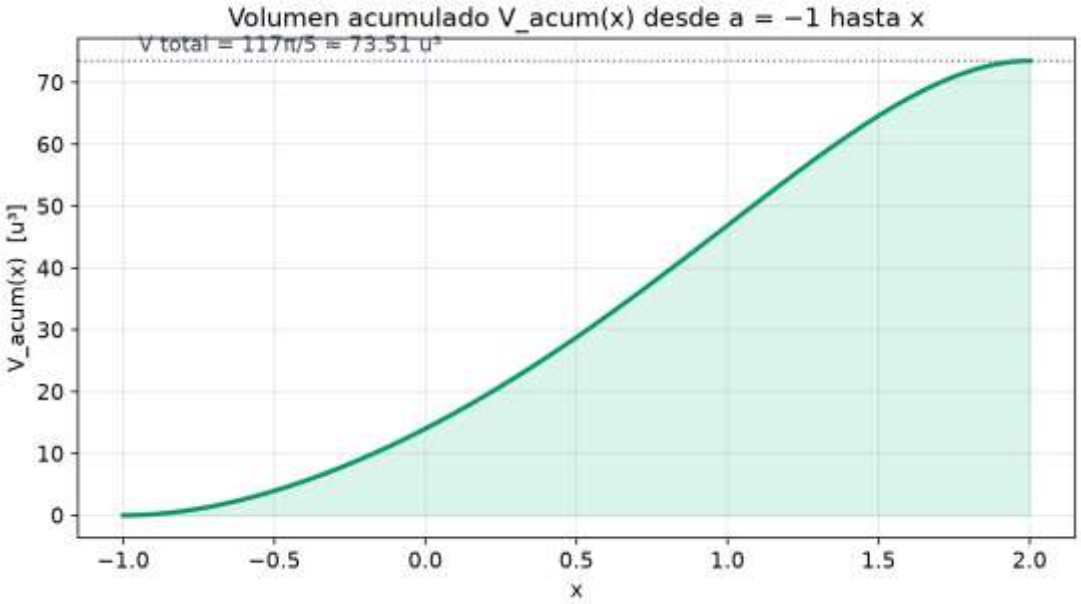


Figura 5. Volumen acumulado V_acum(x) del ejercicio principal.

La curva parte de V_acum(−1) = 0 y termina en V_acum(2) = 73.5133 u³, el volumen total. Es creciente en todo el intervalo porque A(x) ≥ 0. Al inicio crece despacio (en x = −1 las curvas se cortan y A(−1) = 0), alcanza su mayor pendiente en x = 1 —donde la arandela tiene la mayor área, A(1) = 12π ≈ 37.70 u²— y vuelve a aplanarse cerca de x = 2, donde las curvas vuelven a cortarse y A(2) = 0. Esa forma de «S» confirma la relación dV_acum/dx = A(x).

x	f(x)	g(x)	V_acum(x) (u³)
-1.0000	2.0000	2.0000	0.0000
-0.5000	2.5000	1.2500	3.9728
0.0000	3.0000	1.0000	14.0324
0.5000	3.5000	1.2500	28.8045
1.0000	4.0000	2.0000	46.9145
1.5000	4.5000	3.2500	64.6317
2.0000	5.0000	5.0000	73.5133

Tabla 9. Valores seleccionados del archivo exportado datos_solido.csv.

Cerca de la mitad del intervalo (x = 0.5) solo se ha acumulado el 39 % del volumen (28.80 de 73.51 u³), porque las arandelas de la segunda mitad son más grandes que las de la primera.

7.4 Correspondencia con la maqueta

La maqueta reproduce el sólido a escala 1 u = 2 cm. En esa escala, las longitudes se multiplican por 2 y los volúmenes por $2^3 = 8$, de modo que el sólido físico equivale a $73.5133 \times 8 \approx 588.1$ cm³. Sus dimensiones principales son: largo 6 cm (de $a = -1$ a $b = 2$), radio exterior de 4 cm en $x = -1$ y de 10 cm en $x = 2$ (diámetro máximo 20 cm) y radio interior mínimo de 2 cm en $x = 0$. El detalle de construcción se encuentra en el Anexo E y en la guía de construcción de la maqueta.

8. Conclusiones

1. Las curvas $y = x^2 + 1$ y $y = x + 3$ se cortan en $(-1, 2)$ y $(2, 5)$; en $[-1, 2]$ la recta es la función superior (radio exterior) y la parábola la inferior (radio interior). Como la región no toca el eje X, el sólido tiene un hueco y el método de arandelas es el adecuado.
2. El volumen del sólido es $V = \pi\int_{-1}^{2} [(x + 3)^2 - (x^2 + 1)^2]\, dx = 117\pi/5 \approx 73.5133\ u^3$, resultado verificado además con el teorema de Pappus.
3. El simulador reproduce el resultado manual en las cuatro configuraciones con un error menor a 0.0001 %, lo que valida tanto las fórmulas implementadas como el procedimiento manual.
4. El error del simulador se explica completamente por el término de truncamiento de la regla de Simpson, proporcional a h^4; la regla es exacta para polinomios de grado ≤ 3 y con n = 800 el error es despreciable.
5. La gráfica de volumen acumulado confirma el Teorema Fundamental del Cálculo: su pendiente es el área de la sección transversal, máxima en x = 1, donde la curva presenta su punto de inflexión.
6. La maqueta con motor de 5 V permitió observar físicamente el sólido y su hueco central, y su escala (1 u = 2 cm) relaciona directamente las dimensiones físicas con el modelo matemático y la gráfica 3D del simulador.

9. Recomendaciones

1. Verificar siempre cuál función es la superior evaluando un punto de prueba dentro del intervalo, y restar los cuadrados de los radios ($R^2 - r^2$), no el cuadrado de la diferencia.
2. Antes de usar un método, comprobar su compatibilidad con el eje y las condiciones de la región ($f \geq g \geq 0$ para el eje X; $0 \leq a$ para capas alrededor del eje Y).
3. Cuando el integrando no sea suave (por ejemplo, con $\sqrt{x}$ cerca de 0), aumentar el número de subintervalos o comparar dos valores de n para estimar el error numérico.
4. Ampliar el simulador para permitir ejes de rotación desplazados ($y = k$, $x = h$) y el método de capas alrededor del eje X, integrando respecto de y.
5. Para la maqueta, usar materiales ligeros, centrar con precisión la región sobre el eje y balancearla antes de encender el motor para evitar vibraciones; operarla con protección y a distancia segura.
6. Documentar el código y ejecutar las pruebas unitarias después de cada modificación para detectar errores de forma temprana.

10. Bibliografía

Burden, R. L., Faires, J. D., & Burden, A. M. (2017). *Análisis numérico* (10.ª ed.). Cengage Learning.

Chapra, S. C., & Canale, R. P. (2015). *Métodos numéricos para ingenieros* (7.ª ed.). McGraw-Hill Education.

Larson, R., & Edwards, B. H. (2016). *Cálculo* (10.ª ed., Tomo I). Cengage Learning.

Stewart, J., Clegg, D. K., & Watson, S. (2021). *Cálculo de una variable: trascendentes tempranas* (9.ª ed.). Cengage Learning.

Thomas, G. B., Weir, M. D., & Hass, J. (2015). *Cálculo: una variable* (13.ª ed.). Pearson Educación.

Asta, F. (2016). *exp4j: Mathematical expression evaluator* (versión 0.4.8) [Software]. https://www.objecthunter.net/exp4j/

OpenJFX. (2024). *JavaFX 21 documentation*. https://openjfx.io/

Oracle. (2023). *Java SE 21 documentation*. https://docs.oracle.com/en/java/javase/21/

Universidad Mariano Gálvez de Guatemala. (2026). *Proyecto – Emulador de volúmenes de sólidos de revolución: métodos del disco, arandelas y capas cilíndricas* [Enunciado del curso Cálculo II]. Facultad de Ingeniería.

11. Anexos

Anexo A. Código fuente

Se incluye el código completo del motor de cálculo (paquetes model, engine y export), que contiene las ecuaciones, la validación, la integración numérica, la malla 3D y la exportación. Las clases de interfaz gráfica (paquete ui: MainView, Region2DCanvas, AccumCanvas y Solid3DView, ≈ 2 050 líneas) y las pruebas unitarias se entregan en la carpeta del código fuente: src/main/java/com/umg/calculo/.

model/Metodo.java

```
package com.umg.calculo.model;

public enum Metodo {
    DISCOS, ARANDELAS, CAPAS;

    public String etiqueta() {
        return switch (this) {
            case DISCOS -> "Método del Disco";
            case ARANDELAS -> "Método de Arandelas";
            case CAPAS -> "Método de Capas Cilíndricas";
        };
    }
}
```

model/Eje.java

```
package com.umg.calculo.model;

public enum Eje { X, Y }
```

model/Presets.java

```
package com.umg.calculo.model;

import java.util.List;

/** The assignment's fixed test cases: the main exercise plus the 3 required validation configs (Section 13). */
public final class Presets {

    private Presets() {}

    public static final PresetConfig MAIN_PROJECT = new PresetConfig(
            "main_project",
            "Ejercicio Principal (Arandelas - Eje X)",
            "x + 3", "x^2 + 1",
            -1, 2, Metodo.ARANDELAS, Eje.X,
            73.5133,
            "Sólido generado al girar y = x^2 + 1 e y = x + 3 en [-1, 2] alrededor del eje X."
    );

    public static final PresetConfig CONFIG_1 = new PresetConfig(
            "config_1",
            "Configuración 1 (Discos - Eje X)",
            "sqrt(x)", "0",
            0, 4, Metodo.DISCOS, Eje.X,
            25.1327, // pi * [x^2/2]_0^4 = 8*pi
            "f(x) = sqrt(x), g(x) = 0 en [0, 4] rotado en el Eje X."
    );

    public static final PresetConfig CONFIG_2 = new PresetConfig(
            "config_2",
            "Configuración 2 (Arandelas - Eje X)",
            "sqrt(x)", "x^2",
            0, 1, Metodo.ARANDELAS, Eje.X,
            0.9425, // pi * int_0^1 (x - x^4) dx = 0.3*pi
```

```
            "f(x) = sqrt(x), g(x) = x^2 en [0, 1] rotado en el Eje X."
    );

    public static final PresetConfig CONFIG_3 = new PresetConfig(
            "config_3",
            "Configuración 3 (Capas Cilíndricas - Eje Y)",
            "2*x", "x^2",
            0, 2, Metodo.CAPAS, Eje.Y,
            8.3776, // 2*pi * int_0^2 x*(2x - x^2) dx = 2*pi*(4/3) = 8*pi/3 = 8.37758...
            "f(x) = 2x, g(x) = x^2 en [0, 2] rotado en el Eje Y."
    );

    public static final List<PresetConfig> ALL = List.of(MAIN_PROJECT, CONFIG_1, CONFIG_2, CONFIG_3);
}
```

engine/FunctionEvaluator.java

```
package com.umg.calculo.engine;

import net.objecthunter.exp4j.Expression;
import net.objecthunter.exp4j.ExpressionBuilder;

import java.util.function.DoubleUnaryOperator;

/**
 * Compiles a user-typed expression like "x^2 + 1" or "sqrt(x)" into something
 * that can be evaluated at any x. Evaluation failures (domain errors, e.g.
 * sqrt of a negative number) return NaN instead of throwing, so the caller's
 * validation logic can detect and report them uniformly.
 */
public final class FunctionEvaluator implements DoubleUnaryOperator {

    private final Expression expression;
    private final String rawExpression;

    private FunctionEvaluator(Expression expression, String rawExpression) {
        this.expression = expression;
        this.rawExpression = rawExpression;
    }

    /** @throws IllegalArgumentException if the expression has invalid syntax */
    public static FunctionEvaluator compile(String expr) {
        try {
            Expression e = new ExpressionBuilder(expr).variable("x").build();
            return new FunctionEvaluator(e, expr);
        } catch (Exception ex) {
            throw new IllegalArgumentException("Expresión inválida: " + expr, ex);
        }
    }

    @Override
    public double applyAsDouble(double x) {
        try {
            expression.setVariable("x", x);
            double result = expression.evaluate();
            return Double.isFinite(result) ? result : Double.NaN;
        } catch (Exception ex) {
            return Double.NaN;
        }
    }

    public String rawExpression() {
        return rawExpression;
    }

    /**
     * Evaluates a constant typed by the user (limits a, b or the manual
     * volume). Besides plain numbers it accepts a decimal comma ("73,5133"),
     * pi ("117*pi/5", "23.4π") and functions ("sqrt(2)"), so a value copied
     * straight from the hand-written solution can be pasted as-is.
     * Returns NaN if the text is empty, invalid or depends on x.
     */
```

```
    public static double evaluateConstant(String text) {
        if (text == null || text.isBlank()) return Double.NaN;
        String s = text.trim().replace(',', '.').replace("π", "pi");
        try {
            double v = new ExpressionBuilder(s).build().evaluate();
            return Double.isFinite(v) ? v : Double.NaN;
        } catch (Exception ex) {
            return Double.NaN;
        }
    }
}
```

engine/Validator.java

```
package com.umg.calculo.engine;

import com.umg.calculo.model.Eje;
import com.umg.calculo.model.Metodo;

import java.util.ArrayList;
import java.util.List;
import java.util.function.DoubleUnaryOperator;

/** Data-entry validation rules from the assignment (Section 6). */
public final class Validator {

    private Validator() {}

    public record Result(List<String> errores) {
        public boolean esValido() {
            return errores.isEmpty();
        }
    }

    public static Result validar(double a, double b, Metodo metodo, Eje eje,
                                 DoubleUnaryOperator f, DoubleUnaryOperator g) {
        List<String> errores = new ArrayList<>();

        if (Double.isNaN(a) || Double.isNaN(b)) {
            errores.add("Los límites de integración a y b deben ser números válidos.");
        } else if (a >= b) {
            errores.add("Debe cumplirse que a < b.");
        }

        if ((metodo == Metodo.DISCOS || metodo == Metodo.ARANDELAS) && eje != Eje.X) {
            errores.add("Los métodos de discos y arandelas deben ser alrededor del eje X.");
        }
        if (metodo == Metodo.CAPAS && eje != Eje.Y) {
            errores.add("El método de capas cilíndricas debe ser alrededor del eje Y.");
        }
        if (metodo == Metodo.CAPAS && a < 0) {
            errores.add("Para capas cilíndricas alrededor del eje Y debe cumplirse 0 <= a < b.");
        }

        if (!Double.isNaN(a) && !Double.isNaN(b) && a < b) {
            int steps = 200;
            double stepSize = (b - a) / steps;
            boolean invalidFunc = false;
            boolean lowerViolated = false;
            boolean negViolated = false;

            for (int i = 0; i <= steps; i++) {
                double x = a + i * stepSize;
                double fx = f.applyAsDouble(x);
                double gx = g.applyAsDouble(x);

                if (Double.isNaN(fx) || Double.isNaN(gx)) {
                    invalidFunc = true;
                    break;
                }
                if (fx < gx - 1e-9) {
                    lowerViolated = true;
```

```
                }
                if ((metodo == Metodo.DISCOS || metodo == Metodo.ARANDELAS) && gx < -1e-9) {
                    negViolated = true;
                }
            }

            if (invalidFunc) {
                errores.add("Las funciones no se pudieron evaluar en todo el intervalo [a, b].");
            }
            if (lowerViolated) {
                errores.add("Debe cumplirse f(x) >= g(x) en todo el intervalo [a, b].");
            }
            if (negViolated) {
                errores.add("Para discos y arandelas, debe cumplirse f(x) >= g(x) >= 0 en [a, b].");
            }
        }

        return new Result(errores);
    }
}
```

engine/VolumeCalculator.java

```
package com.umg.calculo.engine;

import com.umg.calculo.model.Metodo;

import java.util.function.DoubleUnaryOperator;

/**
 * Numerical core: Simpson's 1/3 rule integration, and the volume integrand
 * for each of the three methods studied in the course. This is the piece the
 * assignment (Section 7) requires to match the manual derivation exactly.
 */
public final class VolumeCalculator {

    private VolumeCalculator() {}

    /** Composite Simpson's rule on [a, b] with n subintervals (bumped up to even if odd). */
    public static double simpsonIntegrate(DoubleUnaryOperator fn, double a, double b, int n) {
        if (a >= b) return 0.0;
        if (n % 2 != 0) n++;
        double h = (b - a) / n;
        double sum = fn.applyAsDouble(a) + fn.applyAsDouble(b);

        for (int i = 1; i < n; i++) {
            double x = a + i * h;
            double coef = (i % 2 == 0) ? 2.0 : 4.0;
            sum += coef * fn.applyAsDouble(x);
        }
        return (h / 3.0) * sum;
    }

    public static double simpsonIntegrate(DoubleUnaryOperator fn, double a, double b) {
        return simpsonIntegrate(fn, a, b, 800);
    }

    /**
     * Builds V(x)'s integrand for the given method:
     *  - Discos:    V = pi * int [f(x)]^2 dx
     *  - Arandelas: V = pi * int ([f(x)]^2 - [g(x)]^2) dx
     *  - Capas:     V = 2*pi * int x * (f(x) - g(x)) dx
     */
    public static DoubleUnaryOperator buildIntegrand(DoubleUnaryOperator f, DoubleUnaryOperator g, Metodo metodo)
{
        return switch (metodo) {
            case DISCOS -> x -> Math.PI * Math.pow(f.applyAsDouble(x), 2);
            case ARANDELAS -> x -> Math.PI * (Math.pow(f.applyAsDouble(x), 2) - Math.pow(g.applyAsDouble(x), 2));
            case CAPAS -> x -> 2 * Math.PI * x * (f.applyAsDouble(x) - g.applyAsDouble(x));
        };
    }
```

```
    /** V_acum(x) = volumen generado desde a hasta x (Section 11). */
    public static double accumulatedVolume(DoubleUnaryOperator integrand, double a, double x) {
        return simpsonIntegrate(integrand, a, x, 200);
    }
}
```

engine/IntersectionFinder.java

```
package com.umg.calculo.engine;

import java.util.ArrayList;
import java.util.List;
import java.util.function.DoubleUnaryOperator;

/**
 * Finds the intersection points of f(x) and g(x), i.e. the roots of
 * h(x) = f(x) - g(x) (part "a" of the main exercise). Scans [lo, hi] on a fine
 * grid looking for sign changes, then refines each one by bisection. Points
 * where f or g are undefined (NaN, e.g. sqrt of a negative) are skipped.
 */
public final class IntersectionFinder {

    public static final double DEFAULT_LO = -10;
    public static final double DEFAULT_HI = 10;
    private static final int SAMPLES = 4000;
    private static final double ZERO_TOL = 1e-12;

    private IntersectionFinder() {}

    /** Region between two consecutive intersections where f(x) >= g(x). */
    public record Interval(double a, double b) {}

    public static List<Double> findIntersections(DoubleUnaryOperator f, DoubleUnaryOperator g) {
        return findRoots(x -> f.applyAsDouble(x) - g.applyAsDouble(x), DEFAULT_LO, DEFAULT_HI, SAMPLES);
    }

    public static List<Double> findRoots(DoubleUnaryOperator h, double lo, double hi, int samples) {
        double step = (hi - lo) / samples;
        List<Double> roots = new ArrayList<>();
        double prevX = Double.NaN, prevH = Double.NaN;

        for (int i = 0; i <= samples; i++) {
            // lo + (hi - lo) * i / samples (not lo + i*step) so grid points like
            // x = 0 land exactly, which catches roots at a NaN boundary (sqrt(x) at 0).
            double x = lo + (hi - lo) * i / samples;
            double hx = h.applyAsDouble(x);
            if (Double.isNaN(hx)) { prevH = Double.NaN; continue; }

            if (Math.abs(hx) < ZERO_TOL) {
                addRoot(roots, x, step);
            } else if (!Double.isNaN(prevH) && Math.abs(prevH) >= ZERO_TOL && Math.signum(prevH) !=
Math.signum(hx)) {
                addRoot(roots, bisect(h, prevX, x), step);
            }
            prevX = x;
            prevH = hx;
        }
        return roots;
    }

    /**
     * First pair of consecutive intersections with f above g between them,
     * or null if there is none (fewer than two intersections, or g above f).
     */
    public static Interval regionBetween(List<Double> roots, DoubleUnaryOperator f, DoubleUnaryOperator g) {
        for (int i = 0; i + 1 < roots.size(); i++) {
            double r1 = roots.get(i), r2 = roots.get(i + 1);
            double mid = (r1 + r2) / 2;
            if (f.applyAsDouble(mid) > g.applyAsDouble(mid)) return new Interval(r1, r2);
        }
        return null;
    }
```

```
    private static double bisect(DoubleUnaryOperator h, double lo, double hi) {
        double hLo = h.applyAsDouble(lo);
        for (int i = 0; i < 100; i++) {
            double mid = (lo + hi) / 2;
            double hMid = h.applyAsDouble(mid);
            if (hMid == 0) return mid;
            if (Math.signum(hMid) == Math.signum(hLo)) { lo = mid; hLo = hMid; } else { hi = mid; }
        }
        return (lo + hi) / 2;
    }

    /** Skips near-duplicates (a flat h can be ~0 on several adjacent grid points). */
    private static void addRoot(List<Double> roots, double x, double step) {
        if (!roots.isEmpty() && x - roots.get(roots.size() - 1) < 1.5 * step) return;
        roots.add(x);
    }
}
```

engine/RevolutionMeshBuilder.java

```
package com.umg.calculo.engine;

import com.umg.calculo.model.Eje;
import com.umg.calculo.model.Metodo;
import javafx.scene.shape.TriangleMesh;
import javafx.scene.shape.VertexFormat;

import java.util.ArrayList;
import java.util.List;
import java.util.function.DoubleUnaryOperator;

/**
 * Builds the solid-of-revolution triangle mesh from a single CLOSED 2D
 * profile (radius, height) that is swept around the axis. Because the
 * profile itself traces outer curve -> end cap -> inner curve -> end cap
 * back to the start (a closed loop), sweeping it a full turn produces one
 * watertight solid — no separate cap/wall pieces needed.
 *
 * Convention (matches the coordinate mapping used by the calculus methods):
 *  - X-axis (discos/arandelas): radius = curve value, height = x.
 *  - Y-axis (capas): radius = x, height = curve value.
 */
public final class RevolutionMeshBuilder {

    private static final int PROFILE_SAMPLES = 60;

    private RevolutionMeshBuilder() {}

    /** One (radius, height) point of the profile, tagged with which curve (f or g) it belongs to. */
    public record ProfilePoint(double r, double h, boolean isCurveF) {}

    public static List<ProfilePoint> buildProfile(DoubleUnaryOperator f, DoubleUnaryOperator g,
                                                  double a, double b, Metodo metodo, Eje eje) {
        int n = PROFILE_SAMPLES;
        List<ProfilePoint> pts = new ArrayList<>(2 * n + 2);

        if (eje == Eje.X) {
            DoubleUnaryOperator rOuter = x -> Math.max(0, f.applyAsDouble(x));
            DoubleUnaryOperator rInner = x -> metodo == Metodo.ARANDELAS ? Math.max(0, g.applyAsDouble(x)) : 0.0;

            pts.add(new ProfilePoint(rInner.applyAsDouble(a), a, false));
            pts.add(new ProfilePoint(rOuter.applyAsDouble(a), a, true));
            for (int i = 1; i <= n; i++) {
                double x = a + (i / (double) n) * (b - a);
                pts.add(new ProfilePoint(rOuter.applyAsDouble(x), x, true));
            }
            pts.add(new ProfilePoint(rInner.applyAsDouble(b), b, false));
            for (int i = n - 1; i >= 1; i--) {
                double x = a + (i / (double) n) * (b - a);
                pts.add(new ProfilePoint(rInner.applyAsDouble(x), x, false));
            }
```

```
        } else {
            for (int i = 0; i <= n; i++) {
                double x = a + (i / (double) n) * (b - a);
                pts.add(new ProfilePoint(x, g.applyAsDouble(x), false));
            }
            pts.add(new ProfilePoint(b, f.applyAsDouble(b), true));
            for (int i = n - 1; i >= 1; i--) {
                double x = a + (i / (double) n) * (b - a);
                pts.add(new ProfilePoint(x, f.applyAsDouble(x), true));
            }
            pts.add(new ProfilePoint(a, f.applyAsDouble(a), true));
        }
        return pts;
    }

    /**
     * @param sweepFraction (0, 1]: 1 = full solid; less = partial reveal, used
     *                      by the sweep-generation animation.
     * @param radialSegments mesh resolution around the axis of revolution.
     */
    public static TriangleMesh buildMesh(List<ProfilePoint> profile, double sweepFraction, int radialSegments) {
        int nPts = profile.size();
        double phiLength = 2 * Math.PI * Math.max(0.001, Math.min(1, sweepFraction));

        float[] points = new float[nPts * (radialSegments + 1) * 3];
        float[] texCoords = new float[nPts * (radialSegments + 1) * 2];

        int pIdx = 0, tIdx = 0;
        for (int j = 0; j <= radialSegments; j++) {
            double theta = (j / (double) radialSegments) * phiLength;
            float cosT = (float) Math.cos(theta);
            float sinT = (float) Math.sin(theta);
            for (ProfilePoint pt : profile) {
                points[pIdx++] = (float) (pt.r() * cosT);
                points[pIdx++] = (float) pt.h();
                points[pIdx++] = (float) (pt.r() * sinT);

                // Samples a 2-color strip texture: near 0.25 = f(x) color,
                // near 0.75 = g(x) color (see Solid3DView's material texture).
                texCoords[tIdx++] = pt.isCurveF() ? 0.25f : 0.75f;
                texCoords[tIdx++] = 0.5f;
            }
        }

        int quadsPerRing = nPts - 1;
        int[] faces = new int[radialSegments * quadsPerRing * 12]; // 2 triangles * 3 verts * 2 (point+texcoord)
idx
        int fIdx = 0;
        for (int j = 0; j < radialSegments; j++) {
            for (int i = 0; i < quadsPerRing; i++) {
                int a0 = j * nPts + i;
                int a1 = j * nPts + i + 1;
                int b0 = (j + 1) * nPts + i;
                int b1 = (j + 1) * nPts + i + 1;

                faces[fIdx++] = a0; faces[fIdx++] = a0;
                faces[fIdx++] = b0; faces[fIdx++] = b0;
                faces[fIdx++] = a1; faces[fIdx++] = a1;

                faces[fIdx++] = a1; faces[fIdx++] = a1;
                faces[fIdx++] = b0; faces[fIdx++] = b0;
                faces[fIdx++] = b1; faces[fIdx++] = b1;
            }
        }

        TriangleMesh mesh = new TriangleMesh(VertexFormat.POINT_TEXCOORD);
        mesh.getPoints().setAll(points);
        mesh.getTexCoords().setAll(texCoords);
        mesh.getFaces().setAll(faces);
        return mesh;
    }

    /** Bounding info used to center the mesh and frame the camera. */
```

```
    public record Bounds(double centerH, double maxExtent) {}

    public static Bounds computeBounds(List<ProfilePoint> profile) {
        double minH = Double.POSITIVE_INFINITY, maxH = Double.NEGATIVE_INFINITY, maxR = 0;
        for (ProfilePoint p : profile) {
            minH = Math.min(minH, p.h());
            maxH = Math.max(maxH, p.h());
            maxR = Math.max(maxR, Math.abs(p.r()));
        }
        double centerH = (minH + maxH) / 2.0;
        double halfHeight = (maxH - minH) / 2.0;
        double maxExtent = Math.max(maxR, halfHeight);
        return new Bounds(centerH, Math.max(maxExtent, 0.5));
    }
}
```

export/DataExporter.java

```
package com.umg.calculo.export;

import java.io.IOException;
import java.io.Writer;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.Locale;

/** Exports the x, f(x), g(x), V_acum(x) sample data required by Section 12. */
public final class DataExporter {

    private DataExporter() {}

    public record Row(double x, double fx, double gx, double vAcum) {}

    public static void exportCsv(Path file, List<Row> rows) throws IOException {
        try (Writer w = Files.newBufferedWriter(file, StandardCharsets.UTF_8)) {
            w.write("x,f_x,g_x,V_acum\n");
            for (Row r : rows) {
                w.write(fmt(r.x()) + "," + fmt(r.fx()) + "," + fmt(r.gx()) + "," + fmt(r.vAcum()) + "\n");
            }
        }
    }

    public static void exportTxt(Path file, List<Row> rows) throws IOException {
        try (Writer w = Files.newBufferedWriter(file, StandardCharsets.UTF_8)) {
            w.write("x\tf(x)\tg(x)\tV_acum\n");
            for (Row r : rows) {
                w.write(fmt(r.x()) + "\t" + fmt(r.fx()) + "\t" + fmt(r.gx()) + "\t" + fmt(r.vAcum()) + "\n");
            }
        }
    }

    private static String fmt(double v) {
        return String.format(Locale.US, "%.4f", v);
    }
}
```

export/ImageExporter.java

```
package com.umg.calculo.export;

import javafx.scene.image.Image;
import javafx.scene.image.PixelReader;
import javafx.scene.image.WritablePixelFormat;

import javax.imageio.ImageIO;
import java.awt.image.BufferedImage;
import java.io.IOException;
import java.nio.file.Path;

/**
 * Saves a JavaFX image as PNG (Section 17: graphs delivered as PNG). Copies the
```

```
 * pixels into an AWT BufferedImage so it doesn't need the extra javafx-swing module.
 */
public final class ImageExporter {

    private ImageExporter() {}

    public static void savePng(Image image, Path file) throws IOException {
        int w = (int) image.getWidth(), h = (int) image.getHeight();
        int[] argb = new int[w * h];
        PixelReader reader = image.getPixelReader();
        reader.getPixels(0, 0, w, h, WritablePixelFormat.getIntArgbInstance(), argb, 0, w);

        BufferedImage out = new BufferedImage(w, h, BufferedImage.TYPE_INT_ARGB);
        out.setRGB(0, 0, w, h, argb, 0, w);
        if (!ImageIO.write(out, "png", file.toFile())) {
            throw new IOException("No hay un escritor PNG disponible.");
        }
    }
}
```

Anexo B. Capturas de pantalla del simulador

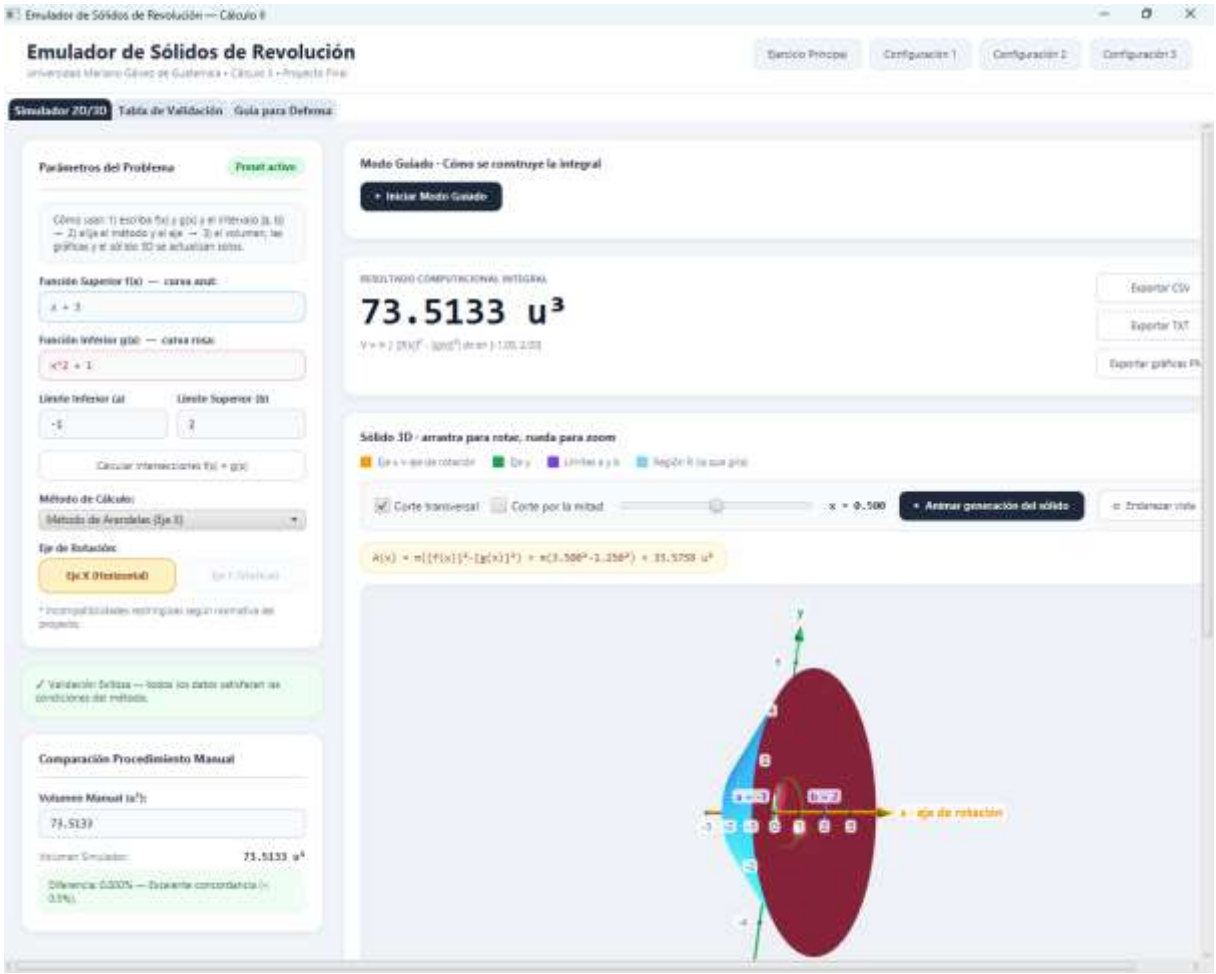


Figura B1. Ejercicio principal: parámetros, validación exitosa, volumen 73.5133 u³, comparación manual (diferencia 0.000 %) y sólido 3D con corte transversal.

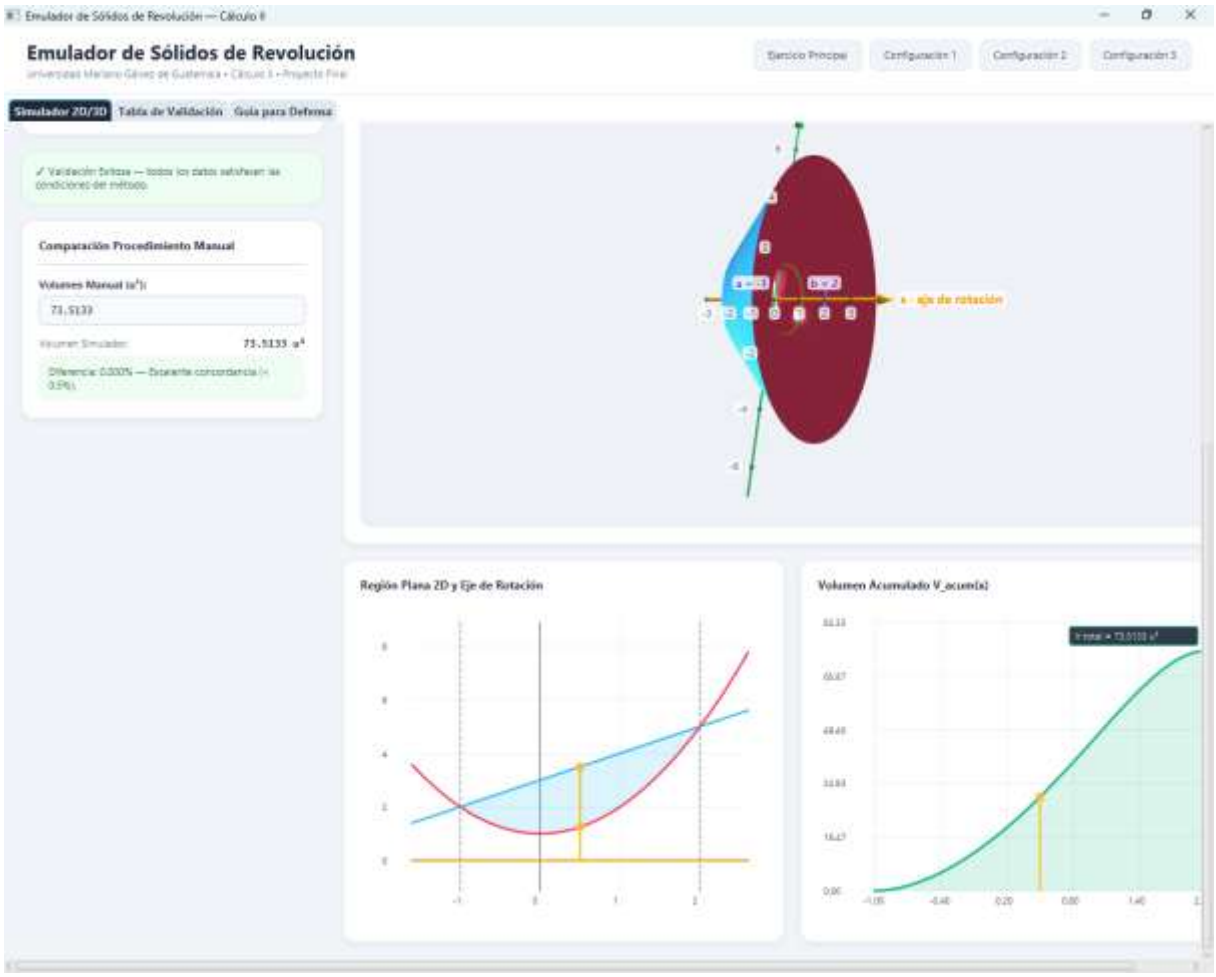


Figura B2. Gráfica 2D de la región con el eje de rotación y gráfica del volumen acumulado V_acum(x).

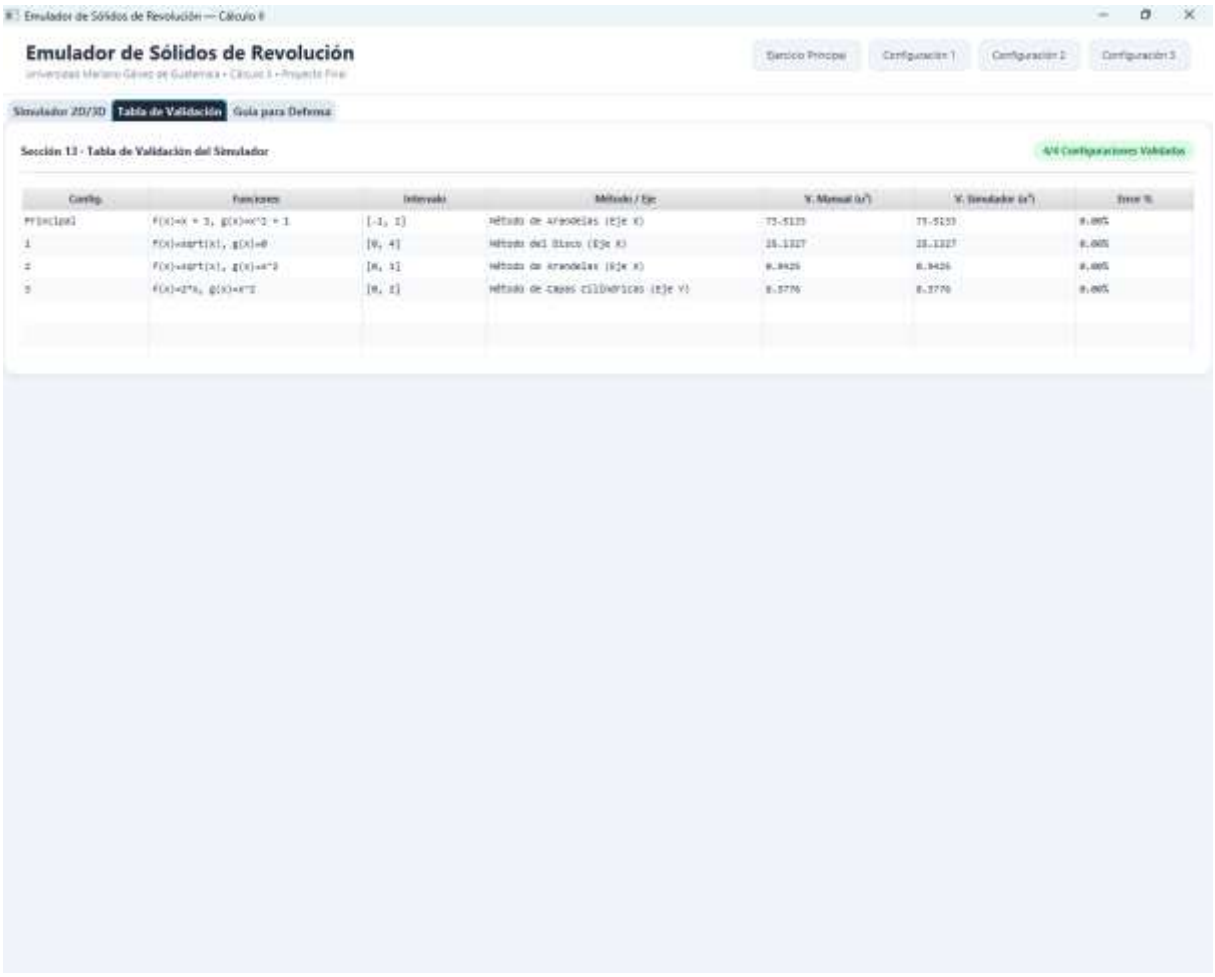


Figura B3. Pestaña «Tabla de Validación»: 4/4 configuraciones validadas.

Anexo C. Archivo de datos exportado

Archivo datos_solido.csv generado para el ejercicio principal (151 filas). Se muestran las primeras y últimas filas; el archivo completo se entrega junto con el informe.

```
x,f_x,g_x,V_acum
-1.0000,2.0000,2.0000,0.0000
-0.9800,2.0200,1.9604,0.0075
-0.9600,2.0400,1.9216,0.0297
-0.9400,2.0600,1.8836,0.0663
-0.9200,2.0800,1.8464,0.1170
-0.9000,2.1000,1.8100,0.1815
-0.8800,2.1200,1.7744,0.2594
-0.8600,2.1400,1.7396,0.3505
-0.8400,2.1600,1.7056,0.4545
-0.8200,2.1800,1.6724,0.5712
-0.8000,2.2000,1.6400,0.7002
…
1.9000,4.9000,4.6100,73.0676
1.9200,4.9200,4.6864,73.2248
1.9400,4.9400,4.7636,73.3492
1.9600,4.9600,4.8416,73.4395
1.9800,4.9800,4.9204,73.4946
2.0000,5.0000,5.0000,73.5133
```

Anexo D. Gráficas

Las gráficas de la región 2D (Figura 2), el sólido 3D (Figura 3), las configuraciones de validación (Figura 4) y el volumen acumulado (Figura 5) se incluyen en el cuerpo del informe y se entregan también en formato PNG.

Anexo E. Maqueta física

La maqueta representa el sólido mediante el efecto de persistencia visual: la región plana R (y su imagen simétrica R′, que equilibra el giro) se recorta a escala 1 u = 2 cm, se pega sobre una lámina transparente fijada a una varilla que hace de eje X, y un motor de corriente directa de 5 V la hace girar. A partir de unas 600 rpm el ojo ya no distingue la lámina en movimiento y percibe el sólido completo, con su hueco central de radio $r(x) = x^2 + 1$.

Requisito del enunciado	Cómo se cumple en la maqueta
Representación del sólido	La región girando a alta velocidad muestra el sólido; complemento: 6 arandelas de cartulina (plantilla B)
Identificación del eje de rotación	Varilla pintada de rojo con banderín «EJE X · rotación»
Región que genera el sólido	Región R recortada en color azul (mismo color del simulador)
Límites de integración	Banderines a = −1 y b = 2 en la base, alineados con los extremos de la región
Dimensiones principales	Largo 6 cm; R = 4 cm en x = −1 y 10 cm en x = 2; r mínimo 2 cm; escala 1 u = 2 cm
Método utilizado	Cartel con «Método de arandelas», la integral y $V = 117\pi/5 \approx 73.51$ u³
Correspondencia con la gráfica 3D	Misma orientación (eje horizontal), mismos colores y mismas proporciones que la vista 3D

Tabla E1. Requisitos de la maqueta (sección 14).

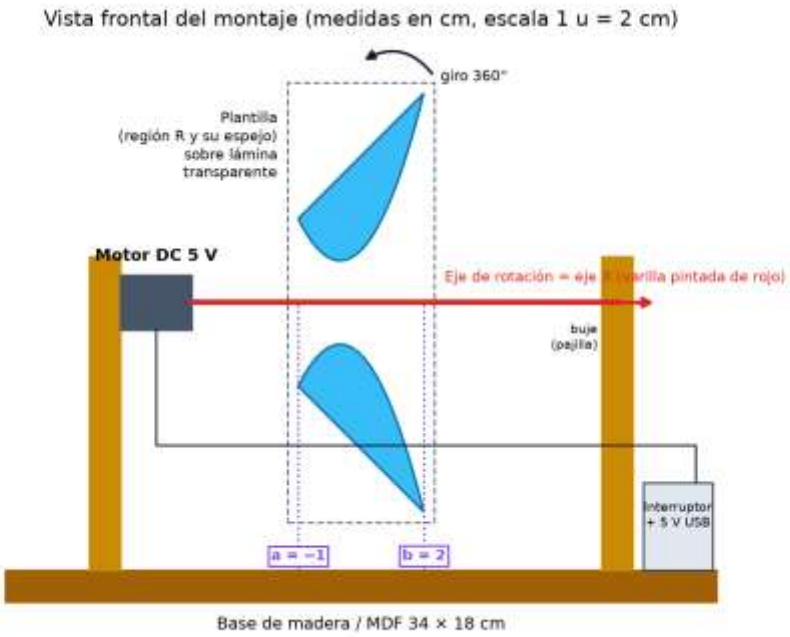


Figura E1. Esquema del montaje de la maqueta.

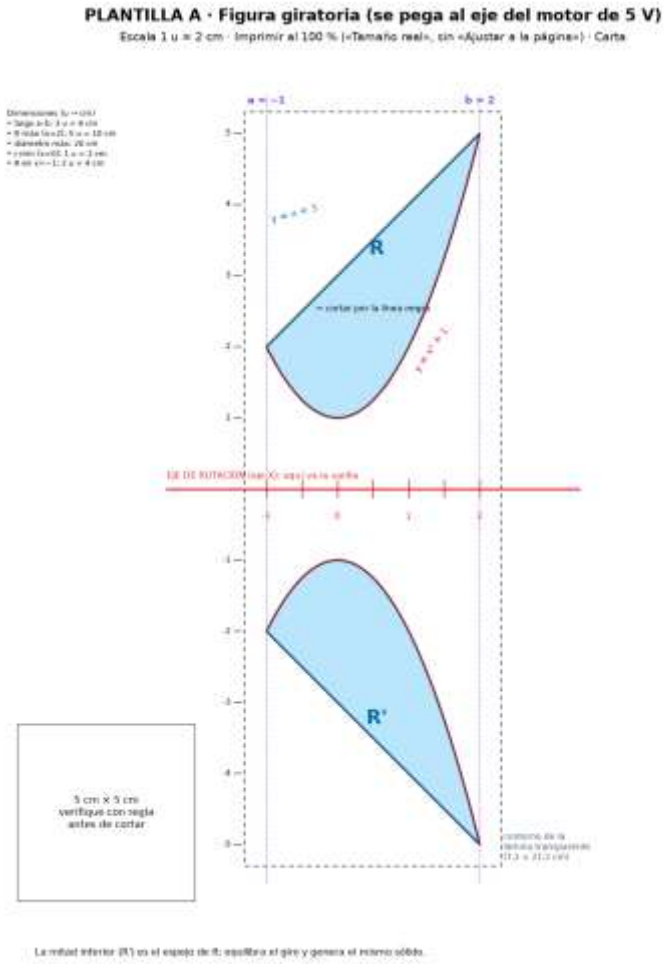


Figura E2. Plantilla de corte de la región (reducida; imprimir a tamaño real desde el PDF de plantillas).

Fotografías de la maqueta

[Insertar aquí la fotografía 1: maqueta detenida, vista lateral, mostrando la región, el eje y los rótulos.]

[Insertar aquí la fotografía 2: maqueta girando, mostrando el sólido de revolución.]

[Insertar aquí la fotografía 3: comparación de la maqueta con la gráfica 3D del simulador.]